A Multiround Asset Market with Midpoint Private News

Shared-State EDSL pilot

EDSL replication study

October 10, 2026

Abstract

We implemented and ran one 30-period call-market session in EDSL's shared-state workflow system. After period 15 settled, a seeded random half of 12 traders received a private analyst estimate that terminal value might be \$16 per share; the actual market redemption remained \$14. All 30 periods settled, but no orders crossed and no shares traded. One treated trader submitted persistent buy orders after the update, but no one offered shares. The pilot validates private information delivery and state persistence, while providing no price response from which to estimate a news effect. The complete machine diagram, Python definition, launch code, results, and response recovery are included.

1 Design

The market follows EDSL's typed call-market example: 30 periods; 12 traders; \$100 cash and four shares per trader; 5% interest on cash; dividends of \$0.40 or \$1.00 with equal probability; marginal-midpoint uniform-price settlement; and \$14 terminal redemption. Traders observe their account, their own history, and the public tape. They submit sealed limit orders in every period. The exchange settles after the order phase.

After the period-15 settlement, six traders selected with Python's seeded shuffle (‘141026‘) received the same private analyst update before the period-16 order phase. The message revised an analyst estimate from \$14 to \$16, and stated that the estimate was uncertain and the actual redemption rule was unchanged. The recipients were trader-00, trader-01, trader-02, trader-05, trader-06, and trader-07. The assignment was held fixed for the whole session.

2 Results

Measure	Pilot result
Trader decisions	360
Settlements	30 / 30
Private update deliveries	6
Completed workflow items	396 / 396
Periods with a transaction	0 / 30
Shares traded	0
Terminal cash, each trader	\$646.34
Terminal shares, each trader	0

There was no transaction price in any period. At period 16, one of six treated traders submitted a buy order; the other five held, and every control trader held. The treated group's mean current-price forecast changed from \$10.85 in period 15 to \$11.02 in period 16. The corresponding control means

were $13.28 and $13.33. These group means are descriptive snapshots from a single session. The treated buyer continued to submit a buy order in later rounds, but no seller appeared. Without transactions, there was no market price adjustment and no spillover through the tape.

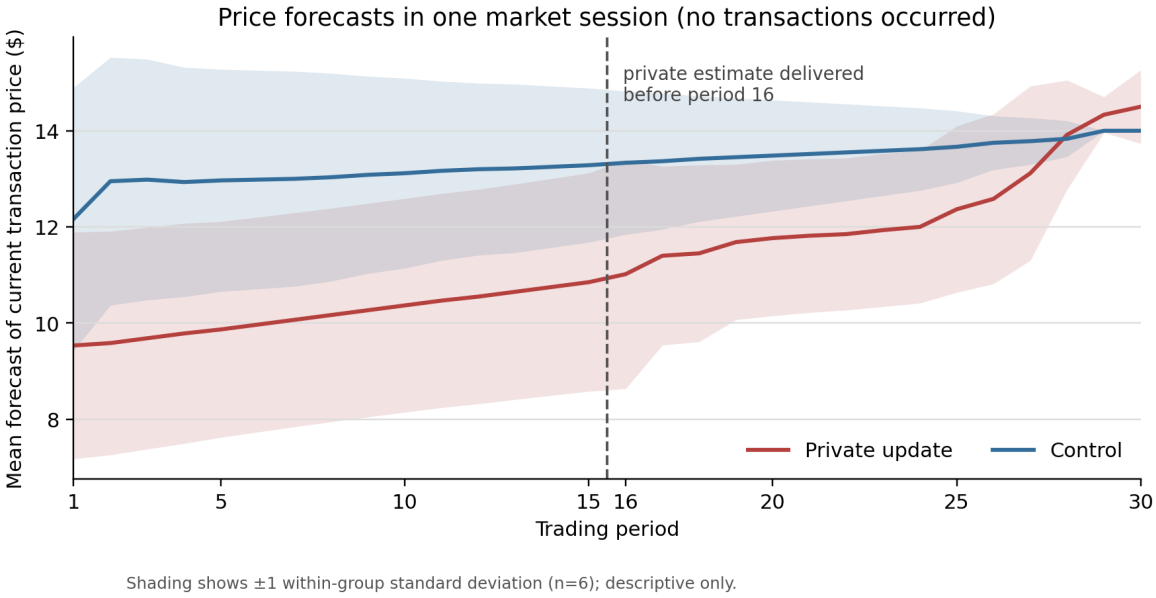


Figure 1: Mean current-price forecast by treatment group. Shading is one within-group standard deviation ($n = 6$ per group); it is descriptive, not an uncertainty interval. The dashed line marks delivery before period 16. No transactions occurred in any period.

The pilot's main result is market inactivity, not a causal estimate of information. The information treatment was successfully delivered and distinguished in private state; it did not create a two-sided market. A useful next design would give traders varied strategic roles or stronger order-choice instructions and repeat the treatment over independently seeded markets.

3 Machine and private view

EDSL's 'call_market' supplies account, order, public tape, period, settlement, interest, dividend, and redemption state. This study extends that machine with one private map and a command that writes an update for one named trader. The read-only view projects only the current trader's message:

```
machine.fields["private_updates"] = state_field(StateType.map(), {})
machine.commands["publish_update"] = Command(
    inputs={"recipient": StateType.text(), "message": StateType.text()},
    effects=(put("private_updates", arg("recipient"), arg("message")),),
)
machine.view["your_update"] = field("private_updates").get(current("name"))
machine.validate()
```

'private_updates' is not exposed as a whole in 'machine.view'. Each trader's rendered prompt reads 'your_update'; the DSL evaluates that projection using the current participant name. A missing map entry renders as "No private analyst update has been issued to you." Thus the treatment is durable shared state, while the view enforces participant-specific disclosure.

The machine diagram below was produced by EDSL's 'Machine.show_diagram' method with the pydot renderer and detailed labels. It shows native market state and the one map, command, and

private projection added for this study.

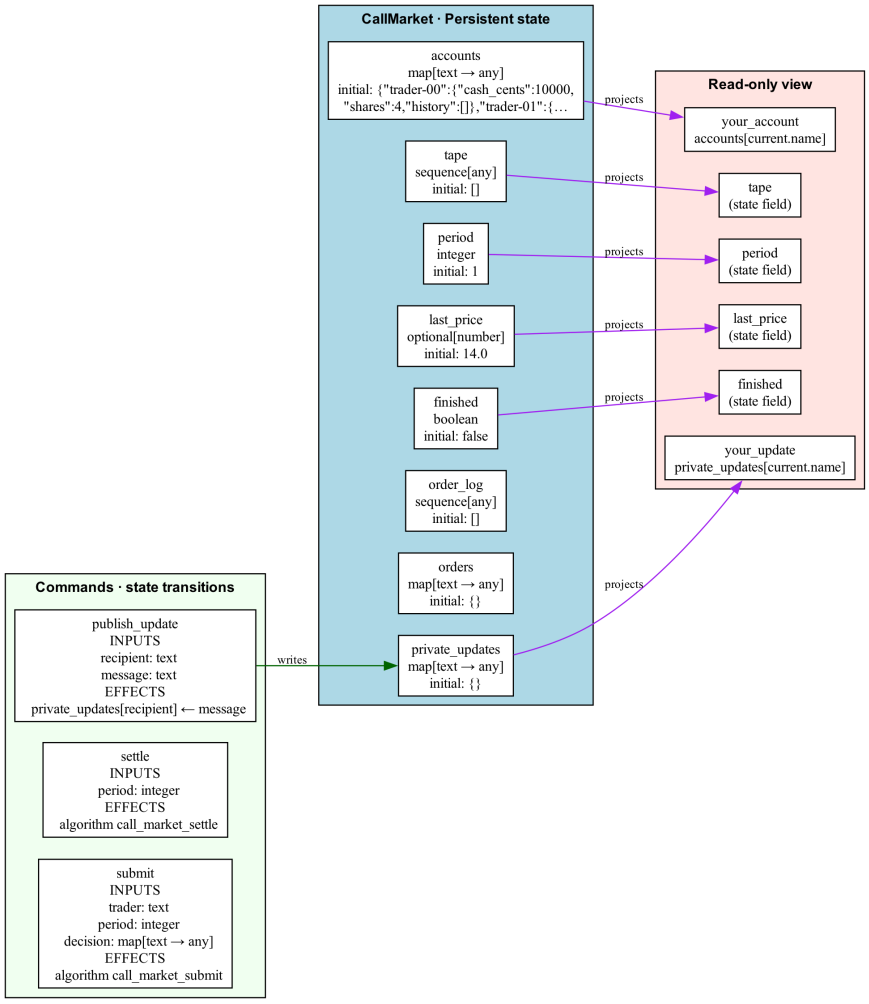


Figure 2: EDSL-generated state-machine diagram for the extended call market.

4 Workflow: one round and the midpoint transition

Each period has a trader order step followed by an exchange settlement step. The order step reads the market and submits each answer as a command. The settlement step runs the deterministic market algorithm. Immediately after period 15 settles, six scripted exchange work items write individual update messages to the private map; period 16 orders depend on those writes. This ordering ensures the treated traders see the update in period 16 and earlier rounds cannot see it.

```
orders = workflow.step(
    f"orders-{period}", Survey([decision]),
    assigned_to=role("trader"), after=previous,
    visible_to=role("exchange"), reads=(market.read(),),
    writes=(market.submit(
        trader=current.agent.name, period=period,
        decision=decision.answer,
    ),),
)
settled = workflow.step(
    f"settle-{period}", settlement_survey,
    assigned_to=role("exchange"), after=orders,
    writes=(market.settle(period=period),),
)
previous = settled
if period == 15:
    for recipient in recipients:
        previous = workflow.step(
            f"publish-private-update-{recipient}", update_survey,
            assigned_to=role("exchange"), after=previous,
            writes=(market.publish_update(
                recipient=recipient, message=UPDATE,
            ),),
        )
```

The update steps are chained, giving every publication an auditable order in the workflow event log. The final period's settlement redeems all remaining shares for $14 and marks the market finished. All 396 work items completed.

5 Launch

The experiment is built from ‘experiment.py‘ and run locally through EDSL with the provider configured in the local environment. Preparation serializes the specification without making model calls:

```
python3 edsl_replication/studies/asset_market_midpoint_news/experiment.py \
  --output edsl_replication/studies/asset_market_midpoint_news/pilot_01 \
  --prepare-only
```

The live pilot used the same command without ‘–prepare-only‘. EDSL's object store uses its default local application-support directory; job, workflow, state, and raw response artifacts are also preserved under ‘pilot_01/‘.

6 Response recovery and interpretation

Four final-period responses returned ‘null‘ for forecasts beyond the terminal horizon. Since the decision schema requires numeric forecasts, those responses were not accepted. The run retained successful

responses and state events. The recovery filled the 12 null out-of-horizon forecast fields with $14, the known terminal redemption amount, and then let the ordinary shared-state workflow settle period 30. The source responses were not overwritten; each filled participant-field pair is recorded in ‘recovery.json‘. These forecasts do not determine the final period’s order. The prompt now makes the terminal forecast rule explicit. No additional model calls were required for recovery.

The pilot contains one market, not independent market replications. Traders share a public tape, so mixed treatment can create interference across traders when trades occur. Here no trades occurred, so this channel did not activate. Forecast comparisons and the lone treated buy order should be read as descriptive evidence about this model, prompt, and session only.